

Національний технічний університет України
«Київський політехнічний інститут імені Ігоря Сікорського»

Факультет інформатики та обчислювальної техніки

Кафедра обчислювальної техніки

Алгоритми та методи обчислень

Лабораторна робота №5

«Розв'язання систем лінійних алгебраїчних рівнянь»

Виконав: студент
групи ІО-42
Куліков М. М.
Номер у списку групи: 9
Перевірив:
Порєв В. М.

Лабораторна робота №5

Тема: «Розв'язання систем лінійних алгебраїчних рівнянь».

Мета: Вивчити алгоритми методів розв'язання систем лінійних алгебраїчних рівнянь на ПК.

Загальне завдання:

- Відповідно до варіанту завдання скласти схему алгоритму розв'язання систем лінійних алгебраїчних рівнянь зазначеним у варіанті методом.
- Відповідно до блок-схеми скласти програму розв'язання систем лінійних алгебраїчних рівнянь алгоритмічною мовою, узгодженою з викладачем.
- Розв'язати СЛАР на комп'ютері відповідно до варіанту

Індивідуальне завдання:

Завдання виконане згідно 9 варіанту.

Таблиця 1 – Завдання згідно варіанту

Номер варіанту	Матриця коефіцієнтів системи	Стовпець вільних членів	Примітка
9 Метод Якобі	2,36 2,37 2,13 2,51 2,40 2,10 2,59 2,41 2,06	1,48 1,92 2,16	$x_1 = 3.864$ $x_2 = -3.899$ $x_3 = 0.753$

Для забезпечення збіжності методу Якобі необхідне виконання умови строгої діагональної переваги матриці ($|a_{ii}| > \sum_{j \neq i} |a_{ij}|$). Оскільки початкова система цій умові не відповідала, було виконано еквівалентне перетворення: до правої і лівої частин кожного i -го рівняння додано величину $3x_i$ (еквівалентне перетворення), що не змінює розв'язок системи, де x_i — наближене значення точного кореня.

Такий підхід не змінює фінальних розв'язків системи, але штучно збільшує елементи головної діагоналі матриці на 3. У результаті перетворення оновлена матриця здобула необхідну діагональну перевагу (наприклад, у першому рядку $5.36 > 2.37 + 2.13$), а її норма зменшилася до $0.9881 < 1$, що забезпечило успішну збіжність ітераційного процесу.

Блок-схеми алгоритмів:

Логіка програми полягає у перевірці матриці на діагональну перевагу та подальшому ітераційному пошуку розв'язку СЛАР. Обчислення виконуються методом Якобі до досягнення заданої точності ε , алгоритм якого представлено на блок-схемі. На кожній ітерації обчислюється новий вектор розв'язку $x^{(k+1)}$ через попередній $x^{(k)}$. Перевірка завершення здійснюється за умовою $\max |x^{(k+1)} - x^{(k)}| < \varepsilon$.

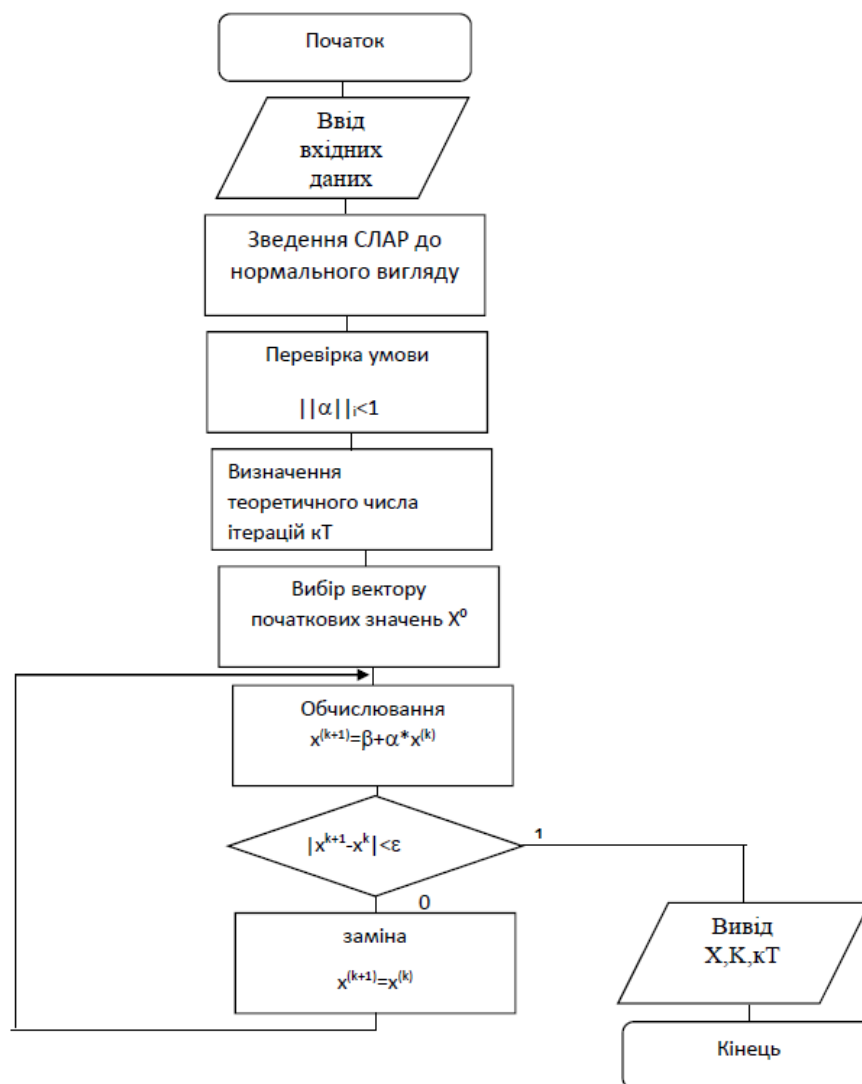


Рисунок 1 – Блок схема методу Якобі

Код програми:

Оскільки програма написана на C# з використанням WPF, і код є громіздким, то вміст усіх файлів представлено в додатку А. У цьому розділі наведений код для реалізації методу Якобі.

Вміст файлу JacobiMath.cs:

```
public class JacobiMath {  
    public bool CheckConvergence(double[,] A, out double maxNorm) {  
        int n = A.GetLength(0);  
        maxNorm = 0;  
        for (int i = 0; i < n; i++) {  
            double sum = 0;  
            for (int j = 0; j < n; j++) { if (i != j) sum += Math.Abs(A[i, j] / A[i, i]); }  
            if (sum > maxNorm) maxNorm = sum;  
        }  
        return maxNorm < 1.0;  
    }  
  
    public List<IterationStep> Solve(double[,] A, double[] B, double epsilon, out double[] finalX) {  
        int n = A.GetLength(0);  
        double[,] alpha = new double[n, n];  
        double[] beta = new double[n];  
        for (int i = 0; i < n; i++) {  
            beta[i] = B[i] / A[i, i];  
            for (int j = 0; j < n; j++) { alpha[i, j] = (i != j) ? -A[i, j] / A[i, i] : 0.0; } }  
        double[] currentX = new double[n];  
        Array.Copy(beta, currentX, n);  
        List<IterationStep> steps = new List<IterationStep>();  
        double delta;  
        int iteration = 0;  
        do {  
            double[] nextX = new double[n];  
            for (int i = 0; i < n; i++) {  
                nextX[i] = beta[i];  
                for (int j = 0; j < n; j++) { nextX[i] += alpha[i, j] * currentX[j]; }  
            }  
            delta = 0;  
            for (int i = 0; i < n; i++) {  
                double d = Math.Abs(nextX[i] - currentX[i]);  
                if (d > delta) delta = d;  
            }  
            steps.Add(new IterationStep { Iteration = iteration + 1, X = (double[])nextX.Clone(), Delta = delta });  
            currentX = nextX;  
            iteration++;  
        } while (delta > epsilon && iteration < 1000);  
        finalX = currentX;  
        return steps;  
    }  
}
```

Результати виконання програми:

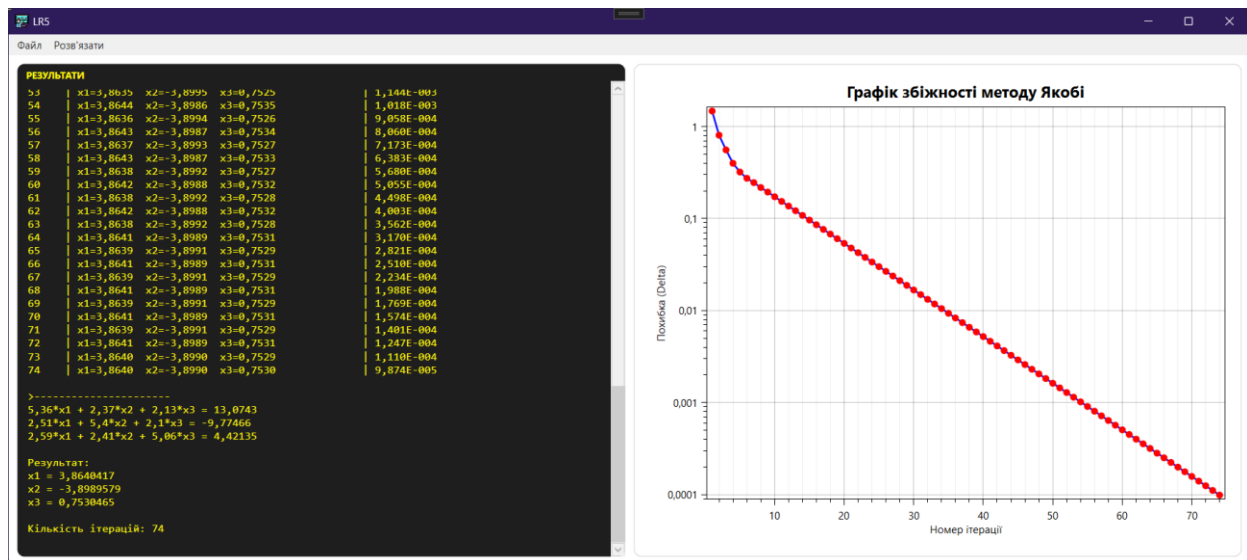


Рисунок 2 – Результат запуску розв’язання СЛАР згідно варіанту

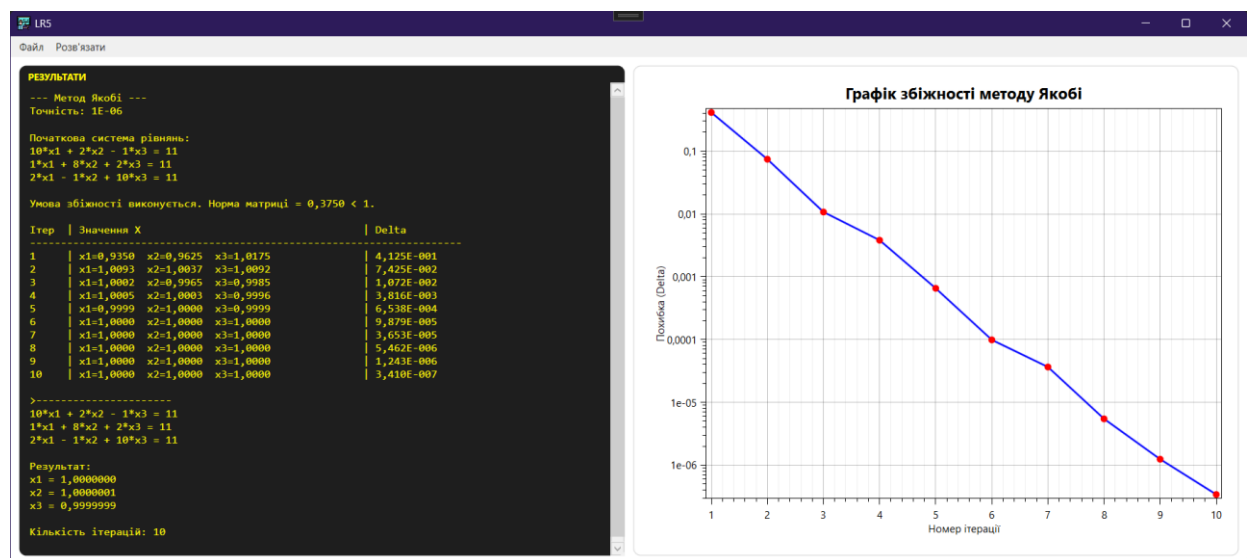


Рисунок 3 – Результат запуску розв’язання іншої СЛАР

Висновки:

У ході роботи реалізовано метод Якобі для розв’язання СЛАР. Для забезпечення збіжності систему було перетворено до вигляду зі строгою діагональною перевагою. У першому експерименті отримано норму матриці $0.9881 < 1$, ітераційний процес збігся за 74 кроки при $\varepsilon = 0.0001$, що свідчить про повільну, але стабільну збіжність. У додатковому експерименті розв’язок досягнуто вже за 10 ітерацій при $\varepsilon = 10^{-6}$, причому похибка зменшувалась монотонно. Це підтверджує залежність швидкості збіжності від властивостей матриці: чим менша норма, тим швидше збігається метод. Отримані результати та графік збіжності узгоджуються з теорією і підтверджують коректність реалізації. Метод Якобі є стабільним, але поступається за швидкістю збіжності методу Гауса–Зейделя.

Додаток А

Вміст файлу MainWindow.xaml:

```
<Window x:Class="LR5.MainWindow"
    xmlns="http://schemas.microsoft.com/winfx/2006/xaml/presentation"
    xmlns:x="http://schemas.microsoft.com/winfx/2006/xaml"
    xmlns:oxy="http://oxyplot.org/wpf"
    Title="LR5" Height="600" Width="1200" WindowStartupLocation="CenterScreen" Icon="/icons8-mathematics-lineal-color-96.png">
    <DockPanel>
        <Menu DockPanel.Dock="Top" Padding="3">
            <MenuItem Header="Файл">
                <MenuItem Header="Зберегти звіт у PDF / Друк..." Click="PrintReport_Click"/>
                <Separator/>
                <MenuItem Header="Вихід" Click="ExitApp"/>
            </MenuItem>
            <MenuItem Header="Розв'язати">
                <MenuItem Header="Ввести дані системи..." Click="RunCustomFunction"/>
            </MenuItem>
        </Menu>

        <Grid Margin="10">
            <Grid.ColumnDefinitions>
                <ColumnDefinition Width="6*"/>
                <ColumnDefinition Width="6*"/>
            </Grid.ColumnDefinitions>

            <Border Grid.Column="0" Background="#1E1E1E" CornerRadius="8" Margin="0,0,5,0">
                <DockPanel>
                    <TextBlock DockPanel.Dock="Top" Text="РЕЗУЛЬТАТИ" Foreground="#FFFFFF906" FontSize="11"
                        Margin="10,5,5,0" FontWeight="Bold"/>
                    <TextBox x:Name="txtLog" Background="Transparent" Foreground="#FFFFFF906"
                        BorderThickness="0" VerticalScrollBarVisibility="Auto" IsReadOnly="True"
                        FontFamily="Consolas" FontSize="13" Padding="10" TextWrapping="Wrap"/>
                </DockPanel>
            </Border>

            <Border Grid.Column="1" Background="White" BorderBrush="LightGray" BorderThickness="1" CornerRadius="8"
                Margin="5,0,0,0">
                <oxy:PlotView x:Name="plotConvergence" Margin="10"/>
            </Border>
        </Grid>
    </DockPanel>
</Window>
```

Вміст файлу MainWindow.xaml.cs:

```
using LR5.Models;
using OxyPlot;
using OxyPlot.Axes;
using OxyPlot.Series;
```

```

using QuestPDF.Fluent;
using QuestPDF.Infrastructure;
using System.Windows;

namespace LR5
{
    public partial class MainWindow : Window
    {
        private readonly JacobiSolver _solver;
        private readonly MatrixSystem _config;
        private AnalysisResult _lastResult;

        public MainWindow()
        {
            InitializeComponent();
            QuestPDF.Settings.License = LicenseType.Community;
            _solver = new JacobiSolver();
            _config = new MatrixSystem();
        }

        private void RunCustomFunction(object sender, RoutedEventArgs e)
        {
            var inputWin = new InputWindow(_config) { Owner = this };

            if (inputWin.ShowDialog() == true)
            {
                txtLog.Clear();
                var math = new JacobiMath();

                if (!math.CheckConvergence(_config.A, out double norm))
                {
                    var result = MessageBox.Show(
                        $"Матриця не задовольняє вимоги збіжності (норма = {norm:F4} >= 1). \n Ітераційний процес розбіжиться. \n \n Впевнені, що хочете продовжити?",
                        "Попередження про збіжність",
                        MessageBoxButton.YesNo,
                        MessageBoxImage.Warning);

                    if (result == MessageBoxResult.No)
                    {
                        txtLog.AppendText("[СКАСОВАНО] Обчислення зупинено користувачем через порушення умови збіжності. \n");
                        return;
                    }
                }

                _lastResult = _solver.Execute(_config.A, _config.B, _config.Epsilon, "Метод Якобі");
                txtLog.Text = _lastResult.LogText;
                txtLog.ScrollToEnd();
                DrawConvergenceGraph(_lastResult.Steps);
            }
        }
    }
}

```

```

}

private void DrawConvergenceGraph(List<IterationStep> steps)
{
    var model = new PlotModel { Title = "Графік збіжності методу Якобі" };

    model.Axes.Add(new LinearAxis
    {
        Position = AxisPosition.Bottom,
        Title = "Номер ітерації",
        MajorGridlineStyle = LineStyle.Solid,
        MinorGridlineStyle = LineStyle.Dot,
        IsZoomEnabled = false,
        IsPanEnabled = false
    });

    model.Axes.Add(new LogarithmicAxis
    {
        Position = AxisPosition.Left,
        Title = "Похибка (Delta)",
        MajorGridlineStyle = LineStyle.Solid,
        IsZoomEnabled = false,
        IsPanEnabled = false
    });

    var series = new LineSeries
    {
        Title = "Зміна похибки",
        Color = OxyColors.Blue,
        MarkerType = MarkerType.Circle,
        MarkerSize = 4,
        MarkerFill = OxyColors.Red
    };

    foreach (var step in steps)
    {
        if (step.Delta > 0)
        {
            series.Points.Add(new DataPoint(step.Iteration, step.Delta));
        }
    }

    model.Series.Add(series);
    plotConvergence.Model = model;
}

private byte[] GetPlotImageBytes()
{
    if (plotConvergence.Model == null) return null;
}

```



```

using (var stream = new System.IO.MemoryStream())
{
    var pngExporter = new OxyPlot.Wpf.PngExporter { Width = 800, Height = 500 };
    pngExporter.Export(plotConvergence.Model, stream);
    return stream.ToArray();
}
}

private void PrintReport_Click(object sender, RoutedEventArgs e)
{
    if (_lastResult == null) return;

    var saveFileDialog = new Microsoft.Win32.SaveFileDialog
    {
        Filter = "PDF файли (*.pdf)|*.pdf",
        FileName = "36im_СЛІА"
    };

    if (saveFileDialog.ShowDialog() == true)
    {
        try
        {
            byte[] chartImage = GetPlotImageBytes();

            QuestPDF.Fluent.Document.Create(container =>
            {
                container.Page(page =>
                {
                    page.Size(QuestPDF.Helpers.PageSizes.A4);
                    page.Margin(1, QuestPDF.Infrastructure.Unit.Centimetre);
                    page.PageColor(QuestPDF.Helpers.Colors.White);
                    page.DefaultTextStyle(x => x.FontSize(12).FontFamily("Times New Roman"));

                    page.Header().PaddingBottom(10).Text("Результати обчислень: Метод Якобі")
                        .SemiBold().FontSize(18).FontColor(QuestPDF.Helpers.Colors.Black);

                    page.Content().Column(column =>
                    {
                        column.Item().PaddingBottom(5).Text("Початкова система рівнянь:").SemiBold();
                        column.Item().Border(1).Padding(5).Column(c =>
                        {
                            int n = _config.A.GetLength(0);
                            for (int i = 0; i < n; i++)
                            {
                                string row = "";
                                for (int j = 0; j < n; j++)
                                {
                                    double val = _config.A[i, j];
                                    row += j == 0 ? $"{val}*x{j + 1}" : $" {(val < 0 ? "-" : "+")} {Math.Abs(val)}*x{j + 1}";
                                }
                                c.Item().Text($"{row} = {_config.B[i]}");
                            }
                        }
                    }
                }
            }
        }
    }
}

```

```

    }
  });

  if (chartImage != null)
  {
    column.Item().PaddingTop(15).Text("Графік збіжності (Delta):").SemiBold();
    column.Item().Image(chartImage, ImageScaling.FitWidth);
  }

  column.Item().PaddingTop(15).Text("Остаточний результат:").SemiBold();
  var finalX = _lastResult.Steps.Last().X;
  column.Item().BorderBottom(1).PaddingVertical(5).Column(c =>
  {
    for (int i = 0; i < finalX.Length; i++)
    {
      c.Item().Text($"x{i + 1} = {finalX[i]:F8}");
    }
    c.Item().Text($"Кількість ітерацій: {_lastResult.Steps.Count}").Italic();
  });

  column.Item().PaddingTop(20).Text("Протокол ітераційного процесу:").SemiBold();
  column.Item().Table(table =>
  {
    int n = _config.A.GetLength(0);
    table.ColumnsDefinition(columns =>
    {
      columns.ConstantColumn(35);
      for (int i = 0; i < n; i++) columns.RelativeColumn();
      columns.RelativeColumn();
    });

    table.Header(header =>
    {
      header.Cell().Element(CellStyle).Text("№");
      for (int i = 0; i < n; i++) header.Cell().Element(CellStyle).Text($"x{i + 1}");
      header.Cell().Element(CellStyle).Text("Delta");

      static QuestPDF.Infrastructure.IContainer CellStyle(QuestPDF.Infrastructure.IContainer container)
        => container.DefaultTextStyle(x => x.SemiBold()).BorderBottom(1).PaddingVertical(5).AlignCenter();
    });

    foreach (var step in _lastResult.Steps)
    {
      table.Cell().Element(ContentStyle).Text(step.Iteration.ToString());
      foreach (var x in step.X) table.Cell().Element(ContentStyle).Text(x.ToString("F5"));
      table.Cell().Element(ContentStyle).Text(step.Delta.ToString("E2"));

      static QuestPDF.Infrastructure.IContainer ContentStyle(QuestPDF.Infrastructure.IContainer container)
        =>
        container.BorderBottom(0.5f).BorderColor(QuestPDF.Helpers.Colors.Grey.Lighten2).PaddingVertical(2).AlignCenter();
    }
  });

```

```

        }
    });
});

});
})
.GeneratePdf(saveFileDialog.FileName);

    MessageBox.Show("Звіт успішно згенеровано!");
}
catch (System.IO.IOException)
{
    MessageBox.Show("Не вдалося зберегти файл. Схоже, він зараз відкритий у якійсь програмі (наприклад, у браузері або PDF-рідері).\n\nБудь ласка, закрийте файл і спробуйте ще раз.",
        "Файл зайнято", MessageBoxButton.OK, MessageBoxImage.Warning);
}
catch (Exception ex)
{
    MessageBox.Show($"Виникла непередбачена помилка під час збереження звіту: \n{ex.Message}",
        "Помилка", MessageBoxButton.OK, MessageBoxImage.Error);
}
}
}
}

private void ExitApp(object sender, RoutedEventArgs e) {
    Application.Current.Shutdown();
} } }

```

Вміст файлу AnalysisResult.cs:

```
using LR5.Models;
```

```
namespace LR5 {
    public class AnalysisResult {
        public string LogText { get; set; }
        public List<IterationStep> Steps { get; set; }
    } }

```

Вміст файлу ILinearSystem.cs:

```
namespace LR5.Models {
    public interface ILinearSystem {
        string Title { get; }
        double[,] A { get; }
        double[] B { get; }
    } }

```

Вміст файлу CustomSystem.cs:

```
namespace LR5.Models {
    public class CustomSystem : ILinearSystem {
        public string Title { get; set; } = "Власна система рівнянь";
        public double[,] A { get; set; }
        public double[] B { get; set; }
    } }

```

Вміст файлу IterationStep.cs:

```
namespace LR5.Models {  
    public class IterationStep {  
        public int Iteration { get; set; }  
        public double[] X { get; set; }  
        public double Delta { get; set; }  
    }  
}
```

Вміст файлу JacobiMath.cs:

```
namespace LR5.Models  
{  
    public class JacobiMath  
    {  
        public bool CheckConvergence(double[,] A, out double maxNorm)  
        {  
            int n = A.GetLength(0);  
            maxNorm = 0;  
  
            for (int i = 0; i < n; i++)  
            {  
                double sum = 0;  
                for (int j = 0; j < n; j++)  
                {  
                    if (i != j) sum += Math.Abs(A[i, j] / A[i, i]);  
                }  
                if (sum > maxNorm) maxNorm = sum;  
            }  
  
            return maxNorm < 1.0;  
        }  
  
        public List<IterationStep> Solve(double[,] A, double[] B, double epsilon, out double[] finalX)  
        {  
            int n = A.GetLength(0);  
  
            double[,] alpha = new double[n, n];  
            double[] beta = new double[n];  
  
            for (int i = 0; i < n; i++)  
            {  
                beta[i] = B[i] / A[i, i];  
                for (int j = 0; j < n; j++)  
                {  
                    alpha[i, j] = (i != j) ? -A[i, j] / A[i, i] : 0.0;  
                }  
            }  
  
            double[] currentX = new double[n];  
            Array.Copy(beta, currentX, n);
```

```

List<IterationStep> steps = new List<IterationStep>();
double delta;
int iteration = 0;

do
{
    double[] nextX = new double[n];
    for (int i = 0; i < n; i++)
    {
        nextX[i] = beta[i];
        for (int j = 0; j < n; j++) { nextX[i] += alpha[i, j] * currentX[j]; }
    }
    delta = 0;
    for (int i = 0; i < n; i++)
    {
        double d = Math.Abs(nextX[i] - currentX[i]);
        if (d > delta) delta = d;
    }
    steps.Add(new IterationStep { Iteration = iteration + 1, X = (double[])nextX.Clone(), Delta = delta });
    currentX = nextX;
    iteration++;
} while (delta > epsilon && iteration < 1000);
finalX = currentX;
return steps;
} } }

```

Вміст файлу JacobiSolver.cs:

```
using System.Text;

namespace LR5.Models
{
    public class JacobiSolver
    {
        public AnalysisResult Execute(double[,] A, double[] B, double epsilon, string title)
        {
            var sb = new StringBuilder();
            sb.AppendLine($"--- {title} ---");
            sb.AppendLine($"Точність: {epsilon}\n");

            int n = A.GetLength(0);

            sb.AppendLine("Початкова система рівнянь.");
            for (int i = 0; i < n; i++)
            {
                for (int j = 0; j < n; j++)
                {
                    double val = A[i, j];
                    if (j == 0)
                    {
                        sb.Append($"{val}*x{j + 1}");
                    }
                }
            }
        }
    }
}
```

```

    }
    else
    {
        string sign = val < 0 ? "-" : "+";
        sb.Append($" {sign} {Math.Abs(val)}*x{j + 1}");
    }
}
sb.AppendLine($" = {B[i]}");
}
sb.AppendLine();

var math = new JacobiMath();

if (!math.CheckConvergence(A, out double norm)) sb.AppendLine($"[УВАГА] Умова збіжності не виконується! Норма матриці = {norm:F4} (має бути < 1).");
else sb.AppendLine($"Умова збіжності виконується. Норма матриці = {norm:F4} < 1.\n");

var steps = math.Solve(A, B, epsilon, out double[] finalX);

sb.AppendLine(string.Format("{0,-5} | {1,-45} | {2,-10}", "Ітер", "Значення X", "Delta"));
sb.AppendLine(new string('-', 70));

foreach (var step in steps) {
    string xStr = "";
    for (int i = 0; i < step.X.Length; i++)
    {
        xStr += $"x{i + 1}={step.X[i]:F4} ";
    }
    sb.AppendLine(string.Format("{0,-5} | {1,-45} | {2,-10:E3}", step.Iteration, xStr.Trim(), step.Delta));
}
sb.AppendLine("\n>-----");
for (int i = 0; i < n; i++) {
    for (int j = 0; j < n; j++)
    {
        double val = A[i, j];
        if (j == 0)
        { sb.Append($" {val}*x{j + 1}"); }
        else {
            string sign = val < 0 ? "-" : "+";
            sb.Append($" {sign} {Math.Abs(val)}*x{j + 1}");
        }
    }
    sb.AppendLine($" = {B[i]}");
}

sb.AppendLine("\nРезультат:");
for (int i = 0; i < finalX.Length; i++)
{ sb.AppendLine($"x{i + 1} = {finalX[i]:F7}"); }
sb.AppendLine($"Кількість ітерацій: {steps.Count}");
return new AnalysisResult { LogText = sb.ToString(), Steps = steps };
} } }

```

Вміст файлу MatrixSystem.cs:

```
namespace LR5.Models
{
    public class MatrixSystem
    {
        public double[,] A { get; set; }
        public double[] B { get; set; }
        public double Epsilon { get; set; }

        public MatrixSystem()
        {
            A = new double[,]{
                { 10.0, 2.0, -1.0 },
                { 1.0, 8.0, 2.0 },
                { 2.0, -1.0, 10.0 }
            };
            B = new double[] { 11.0, 11.0, 11.0 };
            Epsilon = 0.0001;
        }
    }
}
```